Explicacion completa de las lecciones 2.3 y 2.4

La cuadricula base del visor y la cuadricula de componentes BIM

IDEA CENTRAL	React proporciona el punto de montaje; That Open UI organiza la interfaz BIM mediante grids, templates, estados y layouts.

Leccion 2.3: creacion de la cuadricula base del visor

Esta leccion es el primer punto en el que la aplicacion deja de ser unicamente una app de gestion de proyectos y empieza a preparar el espacio BIM.

La idea central es:

ProjectDetailsPage crea un **<bim-grid>** raiz y That Open UI se encarga de organizar dentro de el las distintas zonas del visor.

Archivo principal: **src/react-components/project-details/ProjectDetailsPage.tsx**

1. Que problema resuelve bim-grid

Un visor BIM no es solamente un canvas. Necesita organizar varias zonas:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Sidebar / selector de layouts ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Models ■ ■
■ Queries ■ Viewport 3D ■
■ ■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

El componente **<bim-grid>** permite declarar elementos visuales, definir layouts, cambiar de distribucion, insertar templates y mantener la UI BIM separada de React. Es el sistema de composicion visual de That Open UI.

2. El punto de entrada: ProjectDetailsPage

La parte BIM comienza con una referencia React al grid raiz:

```
const viewerGrid = React.useRef<BUI.Grid<["Main"]>>(null)
```

Esta referencia representa al **<bim-grid>** raiz que aparece en el JSX:

```
<bim-grid ref={viewerGrid} className="viewer-grid"></bim-grid>
```

Hay dos momentos diferentes:

```
Render inicial de React
↓
Se crea fisicamente <bim-grid>
↓
viewerGrid.current apunta al elemento
↓
useEffect ejecuta setupGrid()
↓
That Open configura el grid
```

3. Diferencia entre useRef y BUI.ref

En ProjectDetailsPage se usa la referencia React **ref={viewerGrid}**. En los templates de That Open se usa **${BUI.ref(onCreated)}**. Ambos permiten acceder a un elemento despues de su creacion, pero pertenecen a sistemas diferentes.

```
React JSX
→ ref={viewerGrid}

That Open template
→ ${BUI.ref(onCreated)}
```

4. setupGrid: el puente React → That Open

```
const setupGrid = async () => {
const { current: grid } = viewerGrid
if (!grid) return
const { components, viewport } = await setupComponents()
...
}
```

La funcion obtiene el grid creado por React, inicializa el motor BIM mediante setupComponents() y entrega los objetos BIM al sistema de templates.

```
components
■■■ registro comun de servicios BIM

viewport
■■■ elemento visual donde se dibuja el visor
```

5. El estado inicial de un template

Cada elemento del grid se configura con **template** e **initialState**. El template es una funcion que construye la interfaz y initialState son los datos iniciales que recibe, conceptualmente parecidos a props.

```
React:
<Component components={components} />

That Open:
{
template: componentsGridTemplate,
initialState: { components }
}
```

6. Configuracion actual del grid raiz

En el codigo actual el grid raiz tiene dos elementos: sidebar y componentsGrid. La leccion original hablaba de header, sidebar y componentsGrid, pero la version actual ha evolucionado: el header ha desaparecido, el sidebar utiliza un template real y componentsGrid recibe { components, viewport }.

```
Grid raiz
■■■ sidebar
■■■ componentsGrid
```

7. El layout Main

```
Main: {
template: `
"sidebar" auto
"componentsGrid" 1fr
/1fr
`
}
```

auto significa que la fila ocupa el tamano que necesita su contenido. 1fr significa una fraccion flexible que ocupa el espacio restante. /1fr indica una unica columna flexible.

8. El componentsGrid

El grid raiz solo organiza grandes zonas. El contenido BIM se construye dentro de componentsGridTemplate, que define cuatro elementos:

```
componentsGrid
■■■ viewport
■■■ itemsData
■■■ models
■■■ queries
```

Cada elemento conecta con otro template: viewportContainerTemplate, itemsDataPanelTemplate, modelsPanelTemplate y queriesPanelTemplate.

9. El layout Models

```
Models: {
template: `
"models viewport itemsData" 1fr
"queries viewport itemsData" 1fr
/22rem 1fr 22rem
`
}
```

El viewport ocupa las dos filas porque su nombre se repite. Las columnas laterales tienen 22rem y la columna central ocupa el espacio restante.

```
■■■
```

14. Relacion entre los tres tipos de grid

```
Grid raiz de ProjectDetailsPage
■
■■■ componentsGrid
■
■■■ viewportContainer
■
■■■ viewportGrid flotante
■
■■■ viewerToolbar
```

15. Diferencia entre bim-grid y World

bim-grid organiza la interfaz: paneles, layouts, sidebar, toolbar y viewport-container. World organiza el motor 3D: scene, camera, renderer, raycaster y grid 3D. El bim-grid no renderiza geometria BIM; solo organiza la interfaz que contiene el viewport.

16. Diferencia entre React y los templates de That Open

React utiliza JSX/TSX y un arbol de renderizado React. That Open UI utiliza BUI.html y templates basados en Lit. En ProjectDetailsPage ambos sistemas colaboran: React monta bim-grid y That Open configura sus elementos y layouts.

17. Diferencias entre la leccion y el codigo actual

• El material explica una primera version con header, sidebar y componentsGrid.

• El codigo actual utiliza un sidebar real y layouts Models, Queries y Viewer.

• El evento elementcreated conecta el sidebar con componentsGrid.

• ProjectDetailsPage libera el motor con dispose() al desmontarse.

El Word debe leerse como la explicacion de la construccion progresiva; el repositorio contiene una version posterior y mas completa.

Leccion 2.4: creacion de la cuadricula de componentes BIM

La leccion 2.3 creo el grid general de la pagina BIM. La 2.4 crea un grid interno especializado para organizar el espacio de trabajo BIM.

La diferencia conceptual es:

```
Grid raiz
→ estructura general de la pantalla

ComponentsGrid
→ espacio de trabajo BIM
```

1. El archivo principal

El archivo actual es **src/ui-templates/grids/components/index.ts**. Recibe components y viewport. components permite acceder a los servicios BIM y viewport es el componente visual donde se dibuja el modelo.

2. Declaracion de los elementos

```
grid.elements = {
viewport: { template: viewportContainerTemplate, initialState: { viewport } },
itemsData: { template: itemsDataPanelTemplate, initialState: { components } },
models: { template: modelsPanelTemplate, initialState: { components } },
queries: { template: queriesPanelTemplate, initialState: { components } },
}
```

Cada nombre tiene tres partes: nombre del elemento, template que debe utilizar y estado inicial que recibe.

3. El viewport no se crea aqui

componentsGrid no crea la escena, la camara ni el renderer. Solo coloca el viewport creado previamente en create-world.ts.

```
createWorld()
■■■ crea World
■■■ Scene
■■■ Camera
■■■ Renderer
■■■ Viewport

componentsGrid
■■■ recibe ese Viewport
■■■ lo coloca visualmente
```

4. El layout Models

```
Models: {
template: `
"models viewport itemsData" 1fr
"queries viewport itemsData" 1fr
/22rem 1fr 22rem
`
}
```

La repeticion de viewport hace que ocupe las dos filas. 22rem define los paneles laterales y 1fr define el visor central flexible.

5. Los tres layouts actuales

Aunque la leccion empieza con un unico layout, el codigo actual contiene Models, Queries y Viewer. La ventaja es que los elementos no se duplican: se reutilizan los mismos templates y solo cambia su distribucion.

```
mismos componentes
■
■■■ layout Models
■■■ layout Queries
■■■ layout Viewer
```

6. BUI.ref(onCreated)

```
return BUI.html`
<bim-grid ${BUI.ref(onCreated)} class="components-grid"></bim-grid>
`
```

BUI.ref(onCreated) indica que, cuando el grid exista, debe ejecutarse onCreated y entregarse el elemento creado. Dentro del callback ya se pueden configurar grid.elements, grid.layouts y grid.layout.

7. Por que se comprueba if (!e) return

El callback recibe e?: Element, por lo que e puede ser undefined. La proteccion evita configurar un grid inexistente. Despues se hace casting porque TypeScript conoce un Element generico, pero nosotros sabemos que es un ComponentsGrid.

8. Organizacion de tipos

```
ui-templates/grids/components/
■■■ index.ts
■■■ src/
■■■ index.ts
■■■ types.ts
```

types.ts define el contrato estatico del grid. No crea nada visual. components/index.ts crea y configura el grid en ejecucion.

9. Diferencia entre tipos y ejecucion

```
Tipo TypeScript
≠
Componente visual
```

El tipo ayuda al editor y al compilador a comprobar elementos, layouts y estados; el template es el que crea la interfaz real.

10. Relacion con create-world.ts

```
ProjectDetailsPage
■
▼
setupComponents()
■
▼
createWorld()
■■■ World
■■■ Viewport
■
▼
componentsGridTemplate
■■■ Models panel
■■■ Queries panel
■■■ ItemsData panel
■■■ ViewportContainer
■
▼
Viewport 3D
```

11. Diferencias entre la leccion y el codigo actual

• La leccion empieza con un unico elemento viewport y un unico layout Models.

• El codigo actual ya incluye viewport, models, queries e itemsData.

• El codigo actual dispone de los layouts Models, Queries y Viewer.

• El sidebar descubre dinamicamente los layouts y genera sus botones.

Hay una observacion tecnica importante: el archivo de tipos actual parece parcialmente desactualizado respecto al template ejecutable. El template registra queries, pero la tupla ComponentsGridElements no lo declara; ademas, los layouts Queries y Viewer aparecen en el tipo, aunque no todos los elementos estan representados en esa tupla.

Esto no cambia la idea de la leccion, pero conviene distinguir entre el template ejecutable, que define la configuracion real, y types.ts, que define el contrato estatico que deberia describirla.

12. Idea esencial de la leccion

La leccion 2.4 ensena a organizar el visor por composicion: un grid grande contiene otro grid, que contiene paneles y viewport, que contiene un grid flotante, que contiene la toolbar.

```
Cada nivel tiene una responsabilidad distinta:

Grid raiz → estructura general de la pagina
componentsGrid → area de trabajo BIM
viewportContainer → aloja el viewport
viewportGrid → superpone herramientas
viewerToolbar → acciones de interaccion BIM
```

Resumen conjunto de 2.3 y 2.4

```
React
■■■ ProjectDetailsPage
■■■ bim-grid raiz
■■■ Sidebar
■■■ componentsGrid
■■■ Models
■■■ Queries
■■■ ItemsData
■■■ Viewport
■■■ World
■■■ Scene
■■■ Camera
■■■ Renderer
■■■ modelos BIM
```

La frase resumen es: la leccion 2.3 crea la estructura general del visor y la 2.4 introduce el grid interno que organiza el area de trabajo BIM. React monta la estructura, That Open UI compone la interfaz y el World se encarga del renderizado 3D.